Отчет по отладке приложения Кончев Максим Денисович

Шаг 1: Визуальный осмотр

* Что сделал: визуально осмотрел)

* 1 проблема: Атрибут version в docker-compose.yml устарел, его можно удалить

* Решение: Удалил)

* 2 проблема: Лишняя несуществующая зависимость. Я запускаю на python 3.12, поэтому у меня проблемы при запуске не возникнет, но на python <3.8 будет устанавливаться зависимость с несуществующей версией

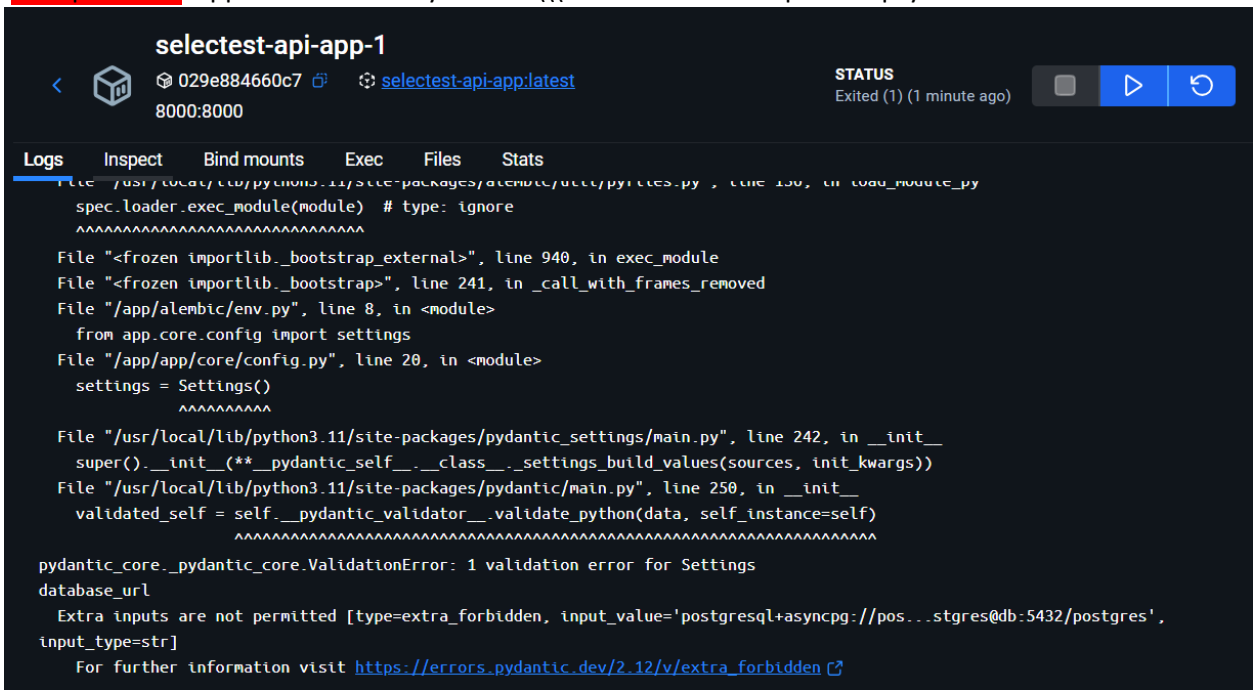
```
fastapi==999.0.0; python_version < "3.8"
```

* Решение: Удалил эту зависимость

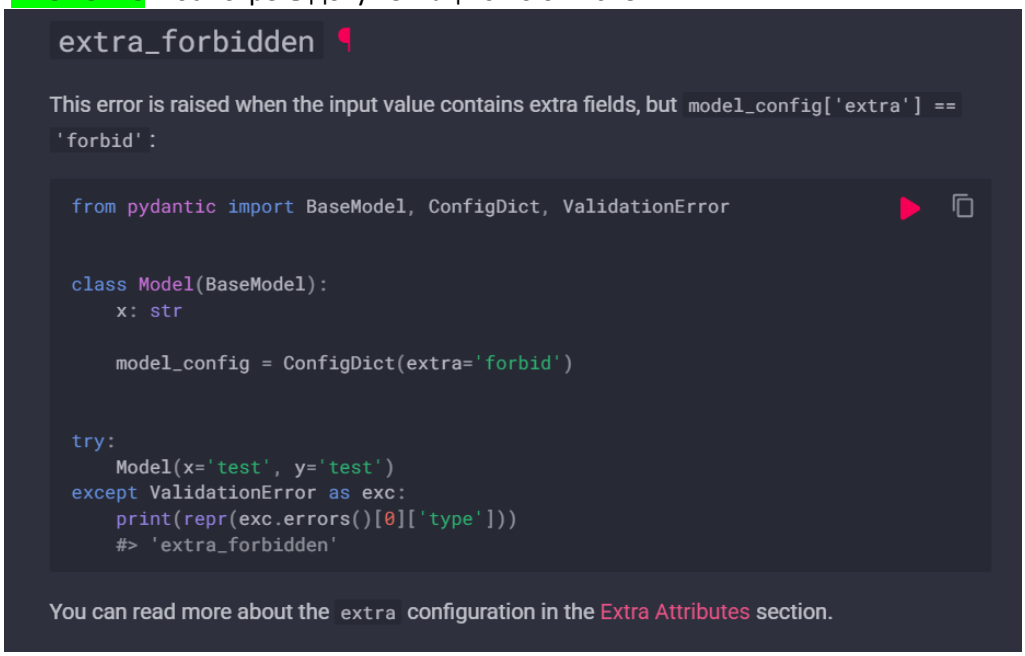
Шаг 2: Анализ и запуск

* Что сделал: Запустил ‘docker-compose up --build’, получил ошибку

* 3 Проблема: app не захотел запускаться (((или же Контейнер не стартует



* Решение: посмотреть документацию по ошибке



Дальше находим, что нам нужно разрешить дополнительные поля

- `'allow'`: Providing extra data is allowed and stored in the `__pydantic_extra__` dictionary attribute:

```python
from pydantic import BaseModel, ConfigDict


class Model(BaseModel):
    x: int

    model_config = ConfigDict(extra='allow')


m = Model(x=1, y='a')
assert m.__pydantic_extra__ == {'y': 'a'}
```

Выше я показал, что умею пользоваться документацией. Не думаю, что необходимо на каждом этапе это показывать.

Разрешаем дополнительные поля

```
app > core > config.py > ...
1   from pydantic import Field
2   from pydantic_settings import BaseSettings, SettingsConfigDict
3
4
5   class Settings(BaseSettings):
6       model_config = SettingsConfigDict(
7           env_file=".env",
8           env_file_encoding="utf-8",
9           case_sensitive=False,
10          extra='allow',
11      )
```

Далее возникла новая проблема

* 4 Проблема: Базы данных с таким именем не существует.

selectest-api\app\core\config.py Строка 14-15

```
app-1 |   File "/usr/local/lib/python3.11/site-packages/asyncpg/connect_utils.py", line 1034, in _connect_addr
app-1 |     return await __connect_addr(params, True, *args)
app-1 |            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
app-1 |   File "/usr/local/lib/python3.11/site-packages/asyncpg/connect_utils.py", line 1102, in __connect_addr
app-1 |     await connected
app-1 | asyncpg.exceptions.InvalidCatalogNameError: database "postgres_typo" does not exist
app-1 exited with code 1
Gracefully Stopping... press Ctrl+C again to force
```

Первая мысль была поменять имя бд postgres_typo на postgres, как указано в .env файле. Поменял, все заработало. Но не понял, почему наша программа подключается по дефолтному значению, если должна брать его из переменной окружения. Не знаю как, но я заметил опечатку в имени переменной)))

* Решение: Поменял DATABSE_URL на DATABASE_URL. Имя БД postgres_typo оставил

UPD: Я вернулся к этому шагу и решил прояснить, почему оставил имя postgres_typo. Я мог изменить это имя на postgres и все бы заработало, даже без редактирования validation_alias переменной, но тут ошибка именно в опечатке в DATABSE_URL. Дефолтный юрл может быть любым, НО на мой взгляд он должен отличаться от DATABASE_URL, которая лежит в .env файле, а если DATABASE_URL(.env)==database_url(config.py), то получается мы сливаем и пользователя и пароль и вообще в чем тогда суть .env файла. В учебном проекте норм и так сделать, но пулять такое в продакшн наверное не очень круто, поэтому я оставил имя postgres_typo, чтобы дефолтное значение database_url(config.py) хоть чем-то отличалось от DATABASE_URL(.env). За лонгрид извините, но я надеюсь вы поняли о чем я.

* 5 Проблема: В случае если у вакансии в поле city стоит null, падает ошибка

```
app-1  | 2026-03-06 06:42:26,890 | INFO | app.main | Запуск приложения
app-1  | 2026-03-06 06:42:26,890 | INFO | app.services.parser | Старт парсинга вакансий
app-1  | 2026-03-06 06:42:27,102 | INFO | httpx | HTTP Request: GET https://api.selectel.ru/proxy/public/employee/api/publ
app-1  | 2026-03-06 06:42:27,103 | ERROR | app.main | Ошибка фонового парсинга: 'NoneType' object has no attribute 'name'
app-1  | Traceback (most recent call last):
app-1  |   File "/app/app/main.py", line 24, in _run_parse_job
app-1  |     await parse_and_store(session)
app-1  |   File "/app/app/services/parser.py", line 43, in parse_and_store
app-1  |     "city_name": item.city.name.strip(),
app-1  |                  ^^^^^^^^^^^^^^
app-1  | AttributeError: 'NoneType' object has no attribute 'name'
```

Ошибка возникает так как в файле parser.py нет обработки значения null у DTO.city Строка 43

app > services > parser.py > parse_and_store

```
25  async def parse_and_store(session: AsyncSession) -> int:
26      logger.info("Старт парсинга вакансий")
27      created_total = 0
28
29      timeout = httpx.Timeout(10.0, read=20.0)
30      try:
31          client = httpx.AsyncClient(timeout=timeout)
32          page = 1
33          while True:
34              payload = await fetch_page(client, page)
35              parsed_payloads = []
36              for item in payload.items:
37                  parsed_payloads.append(
38                      {
39                          "external_id": item.id,
40                          "title": item.title,
41                          "timetable_mode_name": item.timetable_mode.name,
42                          "tag_name": item.tag.name,
43                          "city_name": item.city.name.strip(),
44                          "published_at": item.published_at,
45                          "is_remote_available": item.is_remote_available,
46                          "is_hot": item.is_hot,
47                      }
48                  )
```

* Решение: Добавим проверку на наличие поля city в распаршенных данных

app > services > parser.py > ...

```python
25  async def parse_and_store(session: AsyncSession) -> int:
26      logger.info("Старт парсинга вакансий")
27      created_total = 0
28
29      timeout = httpx.Timeout(10.0, read=20.0)
30      try:
31          client = httpx.AsyncClient(timeout=timeout)
32          page = 1
33          while True:
34              payload = await fetch_page(client, page)
35              parsed_payloads = []
36              for item in payload.items:
37                  if item.city:
38                      item_city = item.city.name.strip()
39                  else:
40                      item_city = None
41                  parsed_payloads.append(
42                      {
43                          "external_id": item.id,
44                          "title": item.title,
45                          "timetable_mode_name": item.timetable_mode.name,
46                          "tag_name": item.tag.name,
47                          "city_name": item_city,
48                          "published_at": item.published_at,
49                          "is_remote_available": item.is_remote_available,
50                          "is_hot": item.is_hot,
51                      }
52                  )
```

* 6 Проблема: Две команды запуска uvicorn в docker-compose.yml и Dockerfile.
Признаюсь, вот тут я решил прибегнуть к помощи дипсика, чтобы разобраться где правильнее оставить команду запуска. Команда в docker-compose переопределяет команду из Dockerfile(я этим фактом никогда до сей поры не пользовался, но возьму на вооружение, спасибо)) и к тому же не будет выполняться пока не применятся миграции. Как будто смысла удалять команду запуска из Dockerfile нет, но и самого смысла в ней нет, если мы для запуска используем docker-compose.yml. Я оставлю и то и то, но комментарием пометил, что я тут был.

* Решение: сомнительное, не знаю как правильно по итогу. Если вы мне напишете это в фидбеке, буду очень рад, дипсику на 100% не верю

```dockerfile
Dockerfile
FROM python:3.11-slim

WORKDIR /app

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

COPY requirements.txt /app/
RUN pip install --no-cache-dir -r requirements.txt

COPY . /app/

#docker-compose doesn't see this command
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

К данному этапу парсер заработал, 25 вакансий оказались в базе данных

* Проблема 7: Фоновая задача парсинга парсит вакансии раз в 5 секунд, но судя по названию переменной в .env файле и Settings, должна делать это раз в 5 минут

```python
app > services > scheduler.py > create_scheduler


def create_scheduler(job: Callable[[], Awaitable[None]]) -> AsyncIOScheduler:
    scheduler = AsyncIOScheduler()
    scheduler.add_job(
        job,
        trigger="interval",
        seconds=settings.parse_schedule_minutes,
        coalesce=True,
        max_instances=1,
    )
    return scheduler

```

* Решение: заменить аргумент функции seconds на minutes

```python
app > services > scheduler.py > create_scheduler


def create_scheduler(job: Callable[[], Awaitable[None]]) -> AsyncIOScheduler:
    scheduler = AsyncIOScheduler()
    scheduler.add_job(
        job,
        trigger="interval",
        minutes=settings.parse_schedule_minutes,
        coalesce=True,
        max_instances=1,
    )
    return scheduler

```

тут поставил для теста 1 минуту, заработало, теперь парсит на минутам

```
app-1  | 2026-03-06 09:03:43,317 | INFO | apscheduler.scheduler | Added job "_run_parse_job" to job s
app-1  | 2026-03-06 09:03:43,317 | INFO | apscheduler.scheduler | Scheduler started
app-1  | INFO:     Application startup complete.
app-1  | INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
app-1  | 2026-03-06 09:04:43,317 | INFO | apscheduler.executors.default | Running job "_run_parse_job
], next run at: 2026-03-06 09:05:43 UTC)" (scheduled at 2026-03-06 09:04:43.316353+00:00)
app-1  | 2026-03-06 09:04:43,317 | INFO | app.services.parser | Старт парсинга вакансий
app-1  | 2026-03-06 09:04:43,514 | INFO | httpx | HTTP Request: GET https://api.selectel.ru/proxy/pub
ancies?per_page=1000&page=1 "HTTP/1.1 200 OK"
app-1  | 2026-03-06 09:04:43,533 | INFO | app.services.parser | Парсинг завершен, новых вакансий: 0
app-1  | 2026-03-06 09:04:43,534 | INFO | apscheduler.executors.default | Job "_run_parse_job (trigge
run at: 2026-03-06 09:05:43 UTC)" executed successfully
```

* Проблема 8: Не знаю можно ли считать это проблемой, но в коде присутствует нное число неиспользуемых импортов в файлах, точно численно не могу сказать, но уверен, что какую-то часть производительности они жрут

* Решение: Удалить неиспользуемые импорты

До

```
app > services > parser.py > ...
import logging
from typing import List

import httpx
from sqlalchemy.ext.asyncio import AsyncSession

from app.core.config import settings
from app.crud.vacancy import upsert_external_vacancies
from app.schemas.external import ExternalVacanciesResponse
```

После

```
app > services > parser.py > ...
import logging

import httpx
from sqlalchemy.ext.asyncio import AsyncSession

from app.crud.vacancy import upsert_external_vacancies
from app.schemas.external import ExternalVacanciesResponse
```

* Проблема 9: Тоже вряд-ли это проблема, но в паре мест в коде длина строк иногда превышала допустимые по pep8 79 символов, но так как какие-то переносы строк все-таки есть, то смею предположить, что в ваших проектах допустимая длина строк 88

* Решение: Буду надеяться, что это исправлять не нужно, и так и было задумано. Если это и в правду была ошибка, посчитайте пожалуйста, что я ее исправил(((

* Проблема 10: Неправльно указан возвращаемый тип в функциях, она же генератор возвращает. Врать в данной ситуации не считаю правильным, поэтому признаюсь, что с нахождением данной ошибки мне помогла IDE VSCode, видно что возвращаемый тип подсвечен в обоих файлах.

app > api > v1 > vacancies.py > get_session

```python
21 async def get_session() -> AsyncSession:
22     async with async_session_maker() as session:
23         yield session
```

app > api > v1 > parse.py > get_session

```python
12 async def get_session() -> AsyncSession:
13     async with async_session_maker() as session:
14         yield session
15
```

* Решение:

Исправляем возвращаемый тип в обоих файлах

app > api > v1 > vacancies.py > list_vacancies_endpoint

```python
1  from typing import List, Optional, AsyncGenerator
2
3  from fastapi import APIRouter, Depends, HTTPException, status
4  from fastapi.responses import JSONResponse
5  from sqlalchemy.ext.asyncio import AsyncSession
6
7  from app.crud.vacancy import (
8      create_vacancy,
9      delete_vacancy,
10     get_vacancy,
11     get_vacancy_by_external_id,
12     list_vacancies,
13     update_vacancy,
14 )
15 from app.db.session import async_session_maker
16 from app.schemas.vacancy import VacancyCreate, VacancyRead, VacancyUpdate
17
18 router = APIRouter(prefix="/vacancies", tags=["vacancies"])
19
20
21 async def get_session() -> AsyncGenerator[AsyncSession, None]:
22     async with async_session_maker() as session:
23         yield session
24
```

app > api > v1 > parse.py > ...

```python
1  from typing import AsyncGenerator
2
3  from fastapi import APIRouter, Depends
4  from sqlalchemy.ext.asyncio import AsyncSession
5
6  from app.db.session import async_session_maker
7  from app.services.parser import parse_and_store
8
9  router = APIRouter(prefix="/parse", tags=["parser"])
10
11
12 async def get_session() -> AsyncGenerator[AsyncSession, None]:
13     async with async_session_maker() as session:
14         yield session
15
```

* Проблема 11:

selectest-api\app\main.py Строка 29,38

Ипользование устаревших on_event вместо современного lifespan(тут мне тоже подсказал VSCode, я применял ранее lifespan в проектах, но не знал, что раньше была какая-то их альтернатива)

```
28
29  @app.on_event("startup")
30  async def on_startup() -> None:
31      logger.info("Запуск приложения")
32      await _run_parse_job()
33      global _scheduler
34      _scheduler = create_scheduler(_run_parse_job)
35      _scheduler.start()
36
37
38  @app.on_event("shutdown")
39  async def on_shutdown() -> None:
40      logger.info("Остановка приложения")
41      if _scheduler:
42          _scheduler.shutdown(wait=False)
43
```

* Решение:

Исправить запуск и остановку, объединить их в один метод lifespan

```
app > main.py > ...
1   import logging
2   from contextlib import asynccontextmanager
3
4   from fastapi import FastAPI
5
6   from app.api.v1.router import api_router
7   from app.core.logging import setup_logging
8   from app.db.session import async_session_maker
9   from app.services.parser import parse_and_store
10  from app.services.scheduler import create_scheduler
11
12  logger = logging.getLogger(__name__)
13
14
15  @asynccontextmanager
16  async def lifespan(app: FastAPI):
17      logger.info("Запуск приложения")
18      await _run_parse_job()
19      global _scheduler
20      _scheduler = create_scheduler(_run_parse_job)
21      _scheduler.start()
22
23      yield
24
25      logger.info("Остановка приложения")
26      if _scheduler:
27          _scheduler.shutdown(wait=False)
28
29  app = FastAPI(title="Selectel Vacancies API", lifespan=lifespan)
30  app.include_router(api_router)
31
32  setup_logging()
```

* Проблема 12: Наличие global переменной.

Вообще использование global переменных вместе с asyncio/threading/multiprocessing не самая хорошая идея, несколько функций могут одновременно принять эту переменную и тогда 🏁Race condition🏁(это из того что я знал сам) как минимум, вообще глобалы это плохо, мне так сказали на Selectel Career Wave).

main.py

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    logger.info("Запуск приложения")
    await _run_parse_job()
    global _scheduler
    _scheduler = create_scheduler(_run_parse_job)
    _scheduler.start()

    yield

    logger.info("Остановка приложения")
    if _scheduler:
        _scheduler.shutdown(wait=False)

app = FastAPI(title="Selectel Vacancies API", lifespan=lifespan)
app.include_router(api_router)
```

* Решение: ~~Хранить app.state~~ ~~Использовать dependency injection~~ Просто убрать глобальную переменную

Вот тут я прибегнул к помощи дипсика и спросил как можно избавиться от global переменной, он сказал использовать app.state, в документации написано, что app.state не используется, вместо него нужно использовать иньекции зависимостей. Не понял к чему это все было, я просто убрал глобальную переменную, все прекрасно работает, она нужна была исключительно ради того, чтобы использовать ее в двух функциях. Наверное можно было это объединить с предыдущей ошибкой, но я вынес в отедльную

```
15  @asynccontextmanager
16  async def lifespan(app: FastAPI):
17      logger.info("Запуск приложения")
18      await _run_parse_job()
19      scheduler = create_scheduler(_run_parse_job)
20      scheduler.start()
21
22      yield
23
24      logger.info("Остановка приложения")
25      if scheduler:
26          scheduler.shutdown(wait=False)
27
28  app = FastAPI(title="Selectel Vacancies API", lifespan=lifespan)
29  app.include_router(api_router)
30
```

* Проблема 13: Приватный метод _run_parse_job()
Во-первых он не должен быть приватным, непонятно зачем он приватный изначально
Во-вторых он почему-то в мейне, хотя это неплохо перекидывается в services/parser.py
* Решение: Сделал метод не приватным, перенес его в services/parser.py
Удалил await _run_parse_job(), тоже непонятно зачем она нужна?

_____Дальше я прям начал докапываться до всего, что я бы исправил_____

* Проблема 14: Хранение чувствительных переменных в docker-compose.yml
* Решение: Указать .env файл

docker-compose.yml Git Local Changes (Working Tree) - 1 of 2 changes

```
3   3     image: postgres:16
4         environment:
5           POSTGRES_USER: postgres
6           POSTGRES_PASSWORD: postgres
7           POSTGRES_DB: postgres
    4     env_file:
    5       - .env
8   6     volumes:
```

* Проблема 15: get_session() в файлах эндпоинтов
* Решение: Перенести get_session в app/db/session.py

app > api > v1 > vacancies.py > ...

```
14  )
15  from app.db.session import get_session
16  from app.schemas.vacancy import VacancyCreate, VacancyRead, VacancyUpdate
17
18  router = APIRouter(prefix="/vacancies", tags=["vacancies"])
19
```

vacancies.py Git Local Changes (Working Tree) - 2 of 2 changes

```
19  19
20
21    async def get_session() -> AsyncGenerator[AsyncSession, None]:
22        async with async_session_maker() as session:
23            yield session
24
```

```
20
21  @router.get("/", response_model=List[VacancyRead])
22  async def list_vacancies_endpoint(
23      timetable_mode_name: Optional[str] = None,
24      city: Optional[str] = None,
25      session: AsyncSession = Depends(get_session),
26  ) -> List[VacancyRead]:
27      return await list_vacancies(session, timetable_mode_name, city)
28
29
```

* Проблема 16: Не особо это наверное проблема, но отсутствует валидация на rtype
Да, тут одно поле и возможно это излишне, но относительно других ручек странно выглядит просто, я бы исправил

```
@router.post("/")
async def parse_endpoint(session: AsyncSession = Depends(get_session)) -> dict:
    created_count = await parse_and_store(session)
    return {"created": created_count}
```

* Решение: Добавить модель ответа для валидации

```


class ExternalParseResponce(BaseModel):
    item_count: int
```

```
router = APIRouter(prefix="/parse", tags=["parser"])


@router.post("/")
async def parse_endpoint(
    session: AsyncSession = Depends(get_session)
) -> schemas.ExternalParseResponce:
    created_count = await parse_and_store(session)
    return {"item_count": created_count}
```

* Проблема 17: Использование List, Optional from typing
Не знаю считать ли это за ошибку, но вместо
Optional[Vacancy] можно писать Vacancy | None
а вместо
List[Vacancy] можно писать list[Vacancy] без импорта библиотеки typing
* Решение: Больше так не делать, наверное. Если можно не использовать, лучше не использовать.

* Проблема 18: Проверки не в своей зоне ответственности.
app/api/v1/vacancies.py строки 35-36, 44-50, 61-62, 71-72

```
@router.post("/", response_model=VacancyRead, status_code=status.HTTP_201_CREATED)
async def create_vacancy_endpoint(
    payload: VacancyCreate, session: AsyncSession = Depends(get_session)
) -> VacancyRead:
    if payload.external_id is not None:
        existing = await get_vacancy_by_external_id(session, payload.external_id)
        if existing:
            return JSONResponse(
                status_code=status.HTTP_200_OK,
                content={"detail": "Vacancy with external_id already exists"},
            )
    return await create_vacancy(session, payload)
```

Эти проверки и ей подобные, это зона ответственности слоя бизнес логики, а слоя бизнес логики нет вообще, у нас api знает о существовании crud, это нарушение ЧА(Чистой Архитектуры) Если делать, то сразу нормально, а если не нормально, то лучше не делать)))
Я наедюсь проверять это задание будет тот же человек, который на CareerWave рассказал мне про канал Тишки, спасибо вам большое. Я думал клинарх мне понятен и вообще это не стоит внимания, но теперь я в нем вообще ничего не понимаю, это заслуживает отдельной дискуссии, но я пока не обладаю компетенциями, чтобы спорить с кем-то наравне. НО даже я с уверенностью говорю, что так как сделано в этом файле, это ту мач, так делать не стоит, исправим)

* Решение: Разнести методы по слоям, попытаться все-таки устроить базовую чистую архитектуру со слоем бизнес логики для вакансий(я не ручаюсь за правильность, но пдфку ЧА Р. Мартина уже нашел)
Я попытался исправить, не учел много проверок, которые можно было бы добавить, но я так понял моя цель исправить ошибки, а не переписать код так, как это сделал бы я. Я бы добавил проверки, ввел бы кастомные exceptions и вообще создавал бы классы для работы со слоем, а не все в одном файле писал.

* Проблема 19: я бы вынес API_URL в .env файл просто потому что не оч красиво такое хардкодить

app > services > parser.py > ...

```python
import httpx
from sqlalchemy.ext.asyncio import AsyncSession

from app.crud.vacancy import upsert_external_vacancies
from app.schemas.external import ExternalVacanciesResponse
from app.db.session import async_session_maker

logger = logging.getLogger(__name__)

API_URL = "https://api.selectel.ru/proxy/public/employee/api/public/vacancies"


async def fetch_page(client: httpx.AsyncClient, page: int) -> ExternalVacanciesResponse:
    response = await client.get(
        API_URL,
        params={"per_page": 1000, "page": page},
    )
```

* Решение: перенести API_URL в .env файл

app > services > parser.py > fetch_page

```python
import logging

import httpx
from sqlalchemy.ext.asyncio import AsyncSession

from app.crud.vacancy import upsert_external_vacancies
from app.schemas.external import ExternalVacanciesResponse
from app.db.session import async_session_maker
from app.core.config import settings

logger = logging.getLogger(__name__)


async def fetch_page(client: httpx.AsyncClient, page: int) -> ExternalVacan
    response = await client.get(
        settings.api_url,
        params={"per_page": 1000, "page": page},
    )
    response.raise_for_status()
    return ExternalVacanciesResponse.model_validate(response.json())
```

```
app > core > config.py > Settings
5   class Settings(BaseSettings):
16      )
17      log_level: str = "INFO"
18      parse_schedule_minutes: int = 5
19      api_url: str = "API_URL"
20
21
22  settings = Settings()
```

По поводу того, что надо было найти 8 ошибок, я честно не знаю нашел ли я все 8, сто процентов просмотрел что-то критичное. Мне говорили, что есть место, в котором API пятисотит, я пытался его найти, но не нашел. Специально спрашивать где оно, для добивания количества я не стал. Мне нечестная победа не нужна))). Шучу, я не позиционирую себя как хороший специалист и уж точно есть ребята, которые заслуживают это место больше меня, либо наравне. Я готов учиться и развиваться столько, сколько положено.

пс Старался не пользоваться сторонними ресурсами, но были места, где не обошлось без дипсика, все места указаны в отчете. Спасибо за классное тестовое